

Regulatory Compliance Engine — Especificação Técnica

Motor de conformidade regulatória para tropicalização de serviços financeiros (EUA → Brasil)

Nota de originalidade: projeto **fictício e original**, desenvolvido do zero para fins de portfólio. Cenário, dados e regras são ilustrativos; não reproduz sistemas, código, arquitetura ou soluções proprietárias de nenhuma empresa. As tecnologias citadas (Go, Fiber, MongoDB, PHP) são de domínio público.

1. A ideia em uma frase

Assim como um produto importado passa pela **alfândega** antes de entrar no país, um **produto financeiro criado sob as regras dos EUA** precisa passar por uma verificação de conformidade antes de operar no Brasil. Este projeto é essa “alfândega regulatória”: recebe uma operação ou a configuração de um produto e responde **“pode operar no Brasil? o que precisa ser adaptado?”**, aplicando regras regulatórias brasileiras de forma configurável.

2. Contexto e problema

Empresas de tecnologia que atendem o setor financeiro convivem com um desafio recorrente: **traduzir norma regulatória em software**. Quando um serviço financeiro concebido no exterior (tipicamente nos EUA) é trazido para o Brasil — o processo de *tropicalização* —, ele precisa se adequar a exigências locais que não existiam na origem: tributação específica, limites, obrigações de comunicação a órgãos reguladores, prevenção à lavagem de dinheiro e proteção de dados.

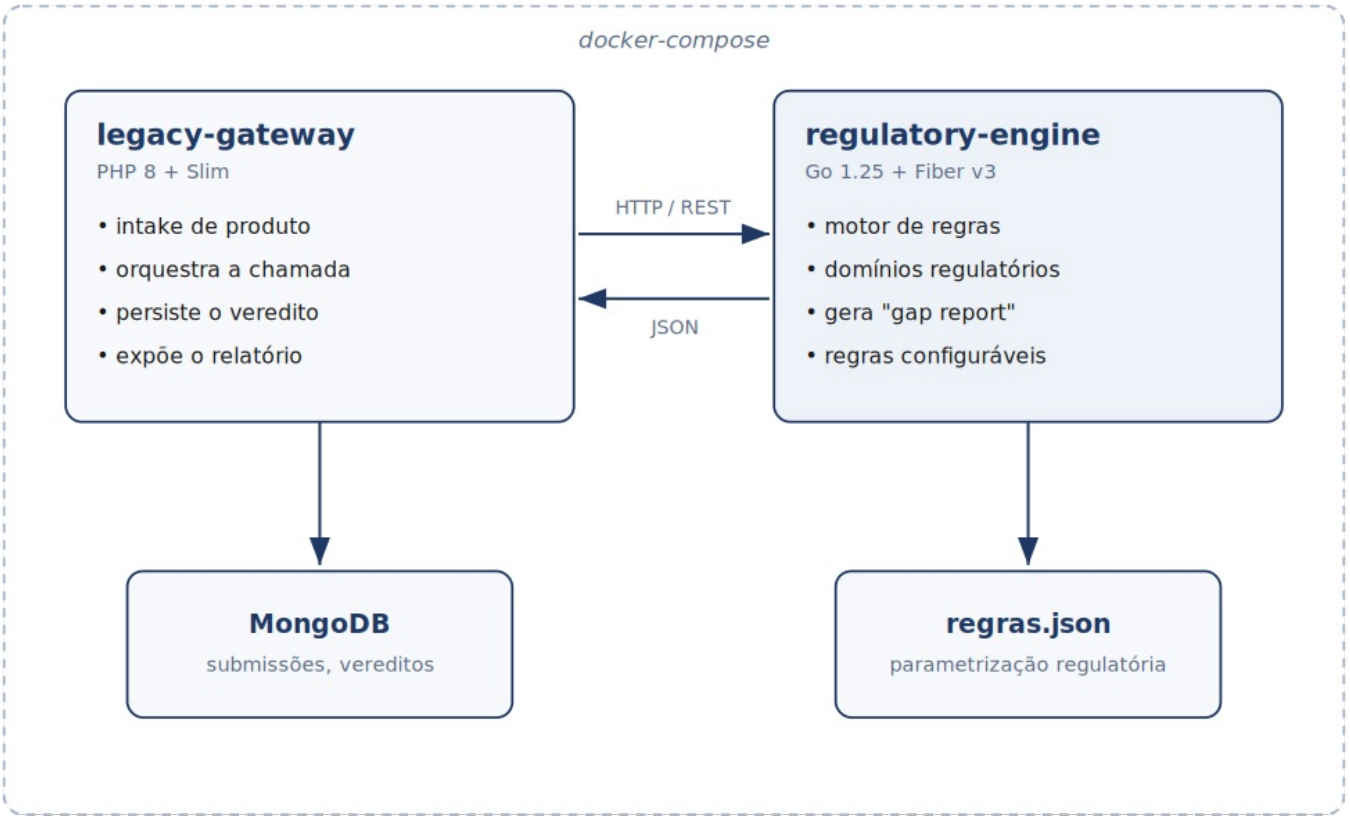
Fazer essa adequação manualmente, produto a produto, é lento e propenso a erro. Este projeto sistematiza a verificação: um serviço central que, dada uma operação, aponta objetivamente as exigências regulatórias brasileiras aplicáveis e as adaptações necessárias.

O projeto demonstra três capacidades:

1. **Entendimento do domínio** financeiro/regulatório brasileiro.
2. **Transformar regra de negócio em software** bem estruturado.
3. **Arquitetura poliglota** com integração entre um sistema legado e uma nova API.

3. Arquitetura

Dois microsserviços containerizados, orquestrados por Docker Compose, reproduzindo um padrão real de migração/modernização gradual — a coexistência entre um sistema legado e uma nova API:



Arquitetura da solução: gateway PHP, motor Go, MongoDB e rules.json orquestrados por Docker Compose

Papéis:

- **regulatory-engine (Go + Fiber):** coração do projeto. Recebe uma operação/produto e devolve o veredito de conformidade. Contém o motor de regras e os domínios regulatórios. Não tem estado de negócio (stateless); lê a parametrização de um arquivo/tabela de regras.
- **Legacy-gateway (PHP + Slim):** representa o sistema legado de uma instituição financeira. Faz o *intake* das submissões, chama o motor Go, persiste o resultado no MongoDB e expõe o histórico e o relatório regulatório. É o ponto de integração legado ↔ moderno.

Decisão de arquitetura-chave: as regras (limites, alíquotas, gatilhos de reporte) **não são hardcoded** — vivem em parametrização configurável (`rules.json` , evoluível para tabela no banco). É uma boa prática essencial no contexto regulatório, porque norma muda com frequência.

4. Domínios regulatórios (escopo inicial)

Sobre os valores: os números abaixo são *ilustrativos* para a estrutura funcionar. Antes de finalizar, confirmar os parâmetros vigentes nas fontes oficiais (Bacen, COAF/UIF, Receita Federal) e ajustar no `rules.json` . A arquitetura configurável existe justamente para isso.

4.1. PLD/FT + COAF (Prevenção à Lavagem de Dinheiro)

- Detecção de operações acima de limite de reporte.
- Marcação de operações **reportáveis ao COAF** (ex.: espécie acima de teto, fracionamento suspeito).
- Screening simples contra lista de **PEP / sancionados** (lista mock local).
- Saída: alertas + indicação de comunicação obrigatória.

4.2. Câmbio / IOF (o core da “tropicalização”)

- Cálculo de **IOF** na entrada de recursos EUA → BR.
- Conversão cambial (taxa parametrizável / mock).
- Regras específicas por tipo de operação (transferência, investimento, empréstimo).
- Saída: tributos aplicáveis + adaptações necessárias vs. o produto original.

4.3. (Roadmap) Limites Bacen/Pix + LGPD

- Fica documentado como próxima fase, para o projeto ter história de evolução.

5. Contrato da API (regulatory-engine)

```
POST /api/v1/evaluate
  → recebe uma operação/produto, retorna o veredito de conformidade + gap report
GET  /api/v1/rules
  → lista as regras carregadas (transparência da parametrização)
GET  /api/v1/health
  → healthcheck
GET  /swagger/*
  → documentação OpenAPI
```

Exemplo de requisição (`POST /api/v1/evaluate`):

```
{
  "product": {
    "type": "personal_loan",
    "origin": "US",
    "origin_currency": "USD"
  },
  "operation": {
    "amount": 75000.00,
    "currency": "USD",
    "method": "international_transfer",
    "counterparty": { "name": "John Doe", "pep": false }
  }
}
```

Exemplo de resposta (gap report):

```
{
  "compliant": false,
  "results": [
    {
      "domain": "IOF",
      "status": "adaptation_required",
      "detail": "Operação sujeita a IOF na entrada; produto original não previa tributação.",
      "calculated_amount": 288.75,
      "reference": "rule:iof.international_transfer"
    },
    {
      "domain": "PLD_COAF",
      "status": "reportable",
      "detail": "Valor acima do teto de comunicação automática ao COAF.",
      "reference": "rule:pld.reporting_threshold"
    }
  ],
  "required_adaptations": [
    "Incluir cálculo e retenção de IOF no fluxo de entrada.",
    "Gerar comunicação ao COAF para a operação."
  ]
}
```

6. Estrutura de pastas (monorepo)

```
regulatory-compliance-engine/
├── docker-compose.yml
├── README.md
├── docs/
│   ├── especificacao_tecnica.md
│   └── arquitetura.png # diagrama (fonte: arquitetura.svg)
├── regulatory-engine/ # Go + Fiber
│   ├── cmd/api/main.go
│   └── internal/
│       ├── engine/ # motor de avaliação
│       ├── domains/ # pld, iof (cada um implementa uma interface Regra)
│       ├── rules/ # carregamento da parametrização
│       ├── httpapi/ # handlers Fiber
│       └── model/ # structs de domínio
│   ├── config/rules.json
│   ├── go.mod
│   └── Dockerfile
├── legacy-gateway/ # PHP + Slim
│   ├── public/index.php
│   ├── src/
│   │   ├── Controller/
│   │   ├── Service/EngineClient.php # integração com o motor Go
│   │   └── Repository/
│   ├── composer.json
│   └── Dockerfile
└── db/
    └── init.js # seed opcional de coleções/índices
```

7. Modelo de dados (MongoDB)

Banco `regulatory`, coleções orientadas a documento (encaixe natural com os payloads JSON):

- **submissions**: cada produto/operação enviado para avaliação — documento com o payload original, origem e data.
- **verdicts**: resultado retornado pelo motor — documento com `compliant`, o array `results` (um por domínio), `required_adaptations` e referência (`submission_id`) à submissão.
- **applied_rules** (opcional): trilha de quais regras dispararam, para auditoria.

A natureza documental do MongoDB casa com o formato do gap report: o veredito é gravado praticamente como a própria resposta da API, sem mapeamento objeto-relacional.

8. Qualidade e demonstração

- **Testes automatizados** no motor Go (unitários por domínio + integração da API), com `testify`.
- **CI/CD** (GitHub Actions): lint + testes + build a cada push/PR.
- **OpenAPI/Swagger** no `regulatory-engine`.
- **Logs estruturados** (JSON) nos dois serviços.
- **README** com: contexto de negócio, diagrama, como rodar (`docker compose up`), exemplos de chamada (`curl`), e o **roadmap** (domínios extras, Grafana/Loki, screening real).
- **Coleção de exemplos** (`curl` / Postman) para rodar em 1 minuto.

9. Roadmap (evolução futura)

1. Domínios adicionais: Limites Bacen/Pix, LGPD, SCR.
2. Observabilidade completa: Prometheus + Grafana + Loki.
3. Regras em banco com versionamento (histórico de vigência das normas).
4. Screening PEP/sancionados contra fonte real.
5. Autenticação (JWT) e trilha de auditoria.